PROJECT HANDBOOK & INTERVIEW PREP · CONDENSED EDITION

Cypress Creek Energy — ML Engineering Internship

Scaling solar forecasting: automated ML operations + forecast accuracy

Jun–Aug 2026 · Quantitative Analytics team · Running in production · Sources: summer-project deck (draft) + resume (the resume is the source of truth)

How to read this handbook

VERIFY A detail inferred, not stated in the deck or resume. Confirm it or drop it.

DEFINE A number whose exact definition or value you must confirm. Interviewers probe these first.

STORY Behavioral scaffold. Fill in real people, moments and outcomes. Never invent them.

SAY Shaded blocks with a left bar are words to say out loud.

Contents

Part 1 — Understand the project 0

1. The internship at a glance 0
2. Business context and the four gaps 0
3. Architecture 0
4. The two pipelines 0
5. The quality gate 0
6. S3 layout redesign 0
7. Release engineering: repo split + Argo CD 0
8. The model 0
9. Cold start as bias vs variance 0
10. Cold start: three bias fixes 0
11. Performance Ratio drift: isolating equipment health 0
12. The truncation study (backtest) 0
13. Adaptive hyperparameter tuning 0
14. Impact: each resume bullet, defended 0
15. Stakeholders 0
16. Status, limitations and next steps 0

Part 2 — Interview prep 0

17. Numbers you can say 0
18. Scripts 0
19. Business-decision questions 0
20. Technical deep dives 0
21. Quick answers: "How did you…?" 0
22. Tough follow-ups 0
23. Behavioral story bank 0
24. The forward-deployed lens and delivery 0

Appendix 0

A. Pre-interview verification checklist 0

B. Confidentiality: what to generalize 0

Diagrams 0

Diagram 1 · System architecture: before and after 0

Diagram 2 · Training pipeline (monthly) 0

Diagram 3 · Inference pipeline (daily) 0

Diagram 4 · Release flow: repo split + Argo CD 0

Diagram 5 · Routing by data availability 0

Diagram 6 · Adaptive tuning: the manifest decision 0

Part 1 — Understand the project

1. The internship at a glance

As an ML Engineering Intern on the Quantitative Analytics team, I took a solar-forecasting library that a developer ran by hand, one site at a time, and made it a **production, Kubernetes-native system for ~450 sites**.

Half	What I built
ML operations	Argo Workflows runs **monthly training** and **daily inference**, one pod per site in parallel. Every model passes an **R^2 / NRMSE gate** and is stored under an *approved* or *rejected* S3 prefix; inference serves approved models only. Hourly 7-day forecasts go to one **Snowflake** table. **Argo CD** redeploys from GitHub.
Forecast accuracy	A **cold-start pipeline** pools in-state mature sites with credibility weighting, so new sites get usable forecasts. A **Performance Ratio (PR) drift** check separates equipment problems from weather and size. **Adaptive tuning** runs only when a site's model is rejected.

Resume outcomes (each defined in §14): ~80% site-management efficiency · +0.23 mean R^2 on cold start · ~75% of anomalies caught · faster approval for ~60–70% of sites · −30% data-fetch latency.

2. Business context and the four gaps

For a solar owner-operator, an hourly generation forecast is an **expected-performance baseline**. Likely consumers VERIFY (name the ones that actually used your table):

Consumer	Use of the 7-day hourly forecast
Asset management / performance engineering	Actual vs expected → spot underperforming sites
Operations & maintenance (O&M)	Schedule maintenance in low-production windows; prioritize truck rolls
Finance	Expected-energy and revenue projections
Energy markets / offtake (where applicable)	Scheduling or bidding merchant / hedged volumes; battery dispatch at co-located storage

The four gaps frame the whole project. Each was a business problem, not just an engineering one:

Gap	Business consequence	Closed by
No automation	Coverage depended on a developer's time; can't scale to ~450 sites	Argo schedules + parallel pods (§4)
No guardrails	A bad model quietly feeds bad forecasts; users can't tell that from real underperformance, so trust erodes	Metric gate, approved/rejected storage, approved-only serving (§5)
Cold start	New sites (often with commissioning issues and the most attention) forecast poorly for ~6 months	Pooled, normalized, credibility-weighted training + PR routing (§9–11)
Static settings	One global hyperparameter set left accuracy on the table	Per-site manifest + targeted search on rejection (§13)

3. Architecture

Diagram 1 · p. 19 **Before:** a developer ran a script per site ID; the library pulled one site from Snowflake, preprocessed, trained and inferred, and saved the model and results to local disk. **After:** Snowflake inputs → monthly training (1 pod per site) → quality gate ($R^2 \geq 0.80$, NRMSE ≤ 15) → S3 per site (`approved/`, `rejected/`, manifest) → daily inference (approved models only) → one Snowflake forecast table. GitHub → Argo CD redeploys both workflows.

Component	Role
Kubernetes	Runs every training and inference job as an isolated pod
Argo Workflows	Cron schedules, a "collect sites" step, fan-out (1 pod per site), conditional flow on metrics, S3 artifacts
Snowflake	**In:** active-site list, weather, generation, site specs (weather *forecasts* at inference). **Out:** one central forecast table
AWS S3	Per site: model (joblib) + metrics/metadata (JSON) under `approved/` or `rejected/`, plus the hyperparameter manifest
Argo CD	GitOps: watches the workflow repo on GitHub and redeploys on change
Two repos	**Library repo:** training + inference logic, published as a versioned package. **Workflow repo:** orchestration + S3 storage

The deck's summary of Argo: it **runs automatically** (on schedule), **works in parallel** (separate pods cut runtime) and **saves deterministically** (artifacts go straight to S3). VERIFY The likely mechanisms: `CronWorkflow` for schedules, a `withParam` loop over the site-list JSON, `when` conditions for the gate, and an S3 artifact repository. Add any `parallelism` cap or `retryStrategy` you configured.

4. The two pipelines

Training — monthly

Diagram 2 · p. 20 Monthly trigger → collect sites → fan-out (1 pod per site) → per-site gate → S3 `approved/` or `rejected/`; a rejected model triggers a targeted search next cycle (§13). Step by step:

1. **Collect sites** from Snowflake. The fleet list is data-driven, so new sites are picked up automatically.
2. **Fan out** one pod per site. Each pod pulls its data, **routes itself by history length** (standard, cold start, or flag, §11), then trains and evaluates.
3. **Write** `model.joblib` + `metrics.json`. VERIFY The JSON likely holds metrics, hyperparameters, data window, library version and route.
4. **Gate** each site. Pass → `{site_id}/approved/`; fail → `{site_id}/rejected/`. Rejected models are kept for diagnosis, and so tuning knows the last decision.

Why monthly: history grows slowly relative to seasonal change, so monthly retraining captures new seasons, degradation and new sites at a fraction of daily compute. VERIFY Your decision, your manager's, or team convention?

Inference — daily

Diagram 3 · p. 21 Daily trigger → collect sites → fan-out (1 pod per site, pulling weather forecasts and site specs). If a model exists in `…/approved/`, load it and append hourly predictions for the next 7 days to the Snowflake table; otherwise that pod fails with the error recorded while every other site keeps running.

- **Approved-only serving.** Inference reads only `{site_id}/approved/`, so a rejected model cannot reach consumers, by construction.
- **Failure isolation.** A site with no approved model fails its own pod with the error recorded; the rest of the fleet finishes.
- **One output table.** All sites append to one Snowflake table, so consumers query one place.
- **Why daily:** weather forecasts update daily, and consumers want a fresh 7-day view.

Two questions interviewers ask VERIFY

(1) When this month's model is rejected, does the site keep serving last month's approved model? If `approved/` still holds it, say "it falls back to the last good model"; otherwise the site has no forecast until approved.

(2) Is the append idempotent on a re-run? Typical answer: a run-date / issue-time column plus a merge, or delete-then-insert per site and issue date.

5. The quality gate

Approve only if $R^2 \geq 0.80$ and NRMSE ≤ 15.0.

Metric	Question it answers	Caveat
R^2	Does the model explain the variation in production?	The daily sun cycle dominates hourly variance, so R^2 is flattered and can't be the only check. Negative R^2 = worse than predicting the mean (one cold-start baseline hit ≈ −0.4).
NRMSE	How big are the errors, in a size-independent unit? One threshold then works across capacities.	VERIFY Know the normalizer: capacity (kWdc) gives small %, mean production gives larger ones, range is a third option. "15.0" means different things under each.

- **The trade-off is coverage vs trust.** Stricter means more sites without a forecast; looser means bad forecasts reach users.
- VERIFY **How thresholds were chosen:** team convention, calibrated so healthy sites pass, or agreed with consumers. DEFINE The typical share of sites passing.
- VERIFY **Evaluation split:** it should be a chronological holdout. A random split leaks adjacent hours.

6. S3 layout redesign

```
s3://<bucket>/<site_id>/
    approved/    model.joblib, metrics.json    <- the only prefix inference reads
    rejected/    model.joblib, metrics.json    <- kept for diagnosis and tuning
    hyperparameter manifest (params + last decision)    <- read at training time
```

Why it removes scans. S3 listing (`ListObjectsV2`) filters by key prefix. With site first, then status, each pod does one small, direct lookup of its own approved artifacts. VERIFY The old layout was likely flat or date-first, so every pod listed a large shared prefix and filtered client-side — work that scales with the whole bucket, repeated by ~450 pods.

Resume claim, −30% data-fetch latency DEFINE Say "end-to-end data-fetch latency per pod, measured before and after on the same run", and know the stage and whether it is a mean or p50.

7. Release engineering: repo split + Argo CD

Diagram 4 · p. 22 **Modeling change:** push the library repo → publish a new package version → bump the version pin in the workflow repo → push the workflow repo. **Orchestration-only change:** push the workflow repo. Either way, Argo CD watches GitHub and syncs Kubernetes, replacing old pods with new ones.

Benefit	Why
Separation of concerns	Modelers change the model without touching scheduling; platform changes need no model release
Explicit, reversible releases	Production runs a **pinned** library version, so rollback = revert the pin
GitOps	Cluster state follows Git; no hand-edited deployments
Low operating cost	A workflow-only change is one push

VERIFY Where the package is published (internal index or artifact store), and whether CI runs tests before publishing.

8. The model

Gradient-boosted trees with XGBoost-style defaults, so say "XGBoost regressor" VERIFY.

```
_DEFAULT_ESTIMATOR_PARAMS = {
    "n_estimators": 500, "max_depth": 3, "learning_rate": 0.05,
    "min_child_weight": 15, "subsample": 0.8, "colsample_bytree": 0.7,
    "reg_alpha": 1.0, "reg_lambda": 3.0, "random_state": 42,
}
```

Deliberately conservative: shallow trees; a low learning rate with many trees; a leaf needs lots of evidence (`min_child_weight` 15); row and column subsampling; L1 + L2 regularization. That suits noisy per-site data, and it is the first rung of adaptive tuning (§13).

Item	What to say
Inputs	Weather, generation, site specs. VERIFY Weather is probably irradiance (GHI/POA), temperature, cloud cover and time-of-day / seasonality; list yours. Site specs: kWdc, `is_tracker`, `tilt_gcr`.
Degradation	Treated identically in both pipelines. VERIFY A feature (e.g., site age) or an assumption?
Target	Production ÷ kWdc in cold start. VERIFY Raw kW or normalized in the standard pipeline?

Watch out: training and serving inputs differ. Training uses *weather data*; inference uses *weather forecasts*. If training used observed weather, production error includes forecast error that the gate never measured. VERIFY Which source training used: historical actuals or archived historical forecasts.

9. Cold start as bias vs variance

The standard pipeline is poor early, improves as data grows, then saturates. Early on, two things hurt: **few data points** (less robust) and **no seasonal coverage** (a model trained on three summer months is biased in winter).

The idea: for roughly the first six months, train on the **target site plus mature sites in the same state**. The phrase to say: *trade specificity for data volume.*

$$\text{Error} = \text{Variance} + \text{Bias}^2 + \text{Irreducible error}$$

The deck writes "+ Bias"; formally it is bias squared.

Move	Effect
Pool in-state mature sites	Many more rows, all seasons → **variance ↓**. But the model learns other sites → **bias ↑**
Normalize target, add site features, weight target rows (§10)	Buys the bias back → **bias ↓**
PR-drift gate (§11)	Drops broken-equipment data that no model could learn → less effective irreducible noise

Assumptions: within-state weather differences are small (in both pipelines), and degradation is treated identically. Site specs are implicit per site in the standard pipeline but become **explicit features** when pooled; temporal patterns are learned across many sites instead of one.

10. Cold start: three bias fixes

Fix	Bias removed	How
1 · Normalize the target	Scale	Predict production ÷ kWdc (a 0–1 fraction of DC capacity), then multiply back. "How many kW?" becomes "what fraction of capacity?", so a 5 MW site's data can inform a 20 MW site.
2 · Site-structure features	Structural	`is_tracker`: single-axis trackers produce a broader daily curve than fixed tilt. `tilt_gcr`: tilt and ground-coverage ratio (the share of land covered by panels), which drives inter-row shading. VERIFY One feature or two?
3 · Credibility-weight target rows	Dilution	With 3 mature sites the new site matters; with 10 it vanishes. Give each target row weight N_{pool} / K, with K = 500 (e.g., `sample_weight` in the fit VERIFY).

Why fix 3 is exactly Bühlmann credibility (derive it if asked)

Mature rows weigh 1 each → total N · n target rows weigh N/K each → total nN/K

Target's share of the loss = (nN/K) ÷ (nN/K + N) = **$n \div (n + K) = Z_{Bühlmann}$**

- **N cancels.** The target's influence doesn't depend on how many mature sites are pooled.
- **It grows smoothly with the site's own history:** n = 100 → ≈17% · n = 500 (= K) → 50% · n = 3,000 → ≈86%.
- **K is the crossover:** the row count at which the site's own data weighs as much as the whole pool. It was set from domain intuition (about 4–5 months), then confirmed with a sweep. VERIFY The rows → months mapping (hourly vs daily rows; are night and filtered hours dropped?). DEFINE Which K values and which metric the sweep compared.
- **The analogy:** actuaries use the same Z to blend a policyholder's own experience with the class average. It is the same problem: little individual data, plenty of related data.

What can't be modeled: post-placed-in-service (PIS) inverter failures and ramp-up instability are neither weather nor structure, so they are flagged instead (§11).

11. Performance Ratio drift: isolating equipment health

PR = avg(production ÷ (kW_{dc} × POA / 1000))

POA = plane-of-array irradiance in W/m²; dividing by 1000 expresses it in "suns"

Quantity	Controls for
production	nothing (raw)
production ÷ kWdc	plant size
production ÷ (kWdc × POA/1000)	plant size **and** weather (how much sunlight arrived)

What's left is the **share of available sunlight converted into energy, which is equipment health**. PR is the standard solar performance metric (IEC 61724-style), so the choice is domain-grounded, not an invented score.

Diagram 5 · p. 23 Every site is routed by history at training time: **6+ months** → standard pipeline; **3–6 months** → cold-start pipeline; **1–2 months** → chronological 80/20 split and ΔPR = PR_test − PR_train. If ΔPR > −0.25 the site goes to cold start; otherwise it is flagged and not trained.

- **Why the threshold is one-sided.** ΔPR > 0 means the site got healthier (e.g., ramp-up finishing), which is safe to train on because pooled data is mostly healthy. A sharp drop means something broke recently, and no training data represents that. *Remove bad data rather than model around it.*

- **Why only 1–2-month sites** VERIFY It is the commissioning window: instability is most likely, and the model has the least data to absorb noise. Was it later extended to mature sites?
- **Resume claim, ~75% of anomalies caught** DEFINE Know the ground truth (outages, O&M tickets, a reviewed list), the denominator, and the false-flag rate.

Limitation	Improvement
An average of hourly ratios is unstable at dawn and dusk	Energy-weighted ratio Σ production ÷ Σ (kWdc × POA/1000), or an irradiance floor. VERIFY Which did you use?
Curtailment, snow and soiling also lower PR	Their signatures differ from failure; a curtailment flag would cut false alarms VERIFY
A fixed −0.25 is a large drop (healthy PR ≈ 0.75–0.85)	Tune it against labeled incidents

12. The truncation study (backtest)

Design: simulate new sites without leakage.

1. Take a **mature** site and truncate it to its first 1…6 months, so it pretends to be new.
2. Draw in-state pool data from a **two-year window ending at the target's last date after truncation**, so the pool never sees the future.
3. Compare **baseline vs credibility-weighted R^2** at each truncation length. VERIFY Is the baseline the target-only standard pipeline, or pooling without the weighting? It changes what the lift means.

Results: 18 sites in IL, NC and OR (approximate chart readings):

Pattern	Reading	Meaning
Months 1–2	Several OR sites rise from ~0 or negative R^2 to ≈0.5–0.7; one baseline ≈ −0.4	Gains concentrate exactly where cold start matters
NC sites	≈0.95 either way	Pooling doesn't hurt healthy sites; the weighting protects them
Months 3–6	The two lines mostly converge	Expected: Z → 1 as the site's own data grows
Some IL sites	Dip mid-window in *both* lines (≈0.15–0.45); one recovers only with weighting	Site or season effects, not the method. VERIFY Equipment (the PR story) or winter/snow in the test window?

Resume claim, +0.23 mean R^2 DEFINE Over which sites, at which truncation months, and whether it is the mean of per-site differences. It is credible *because* the effect sits in the early months, so never let it sound like "+0.23 at every site at all times."

13. Adaptive hyperparameter tuning

Problem: one default configuration for every site plateaus as data drifts. For non-technical audiences: *"The chef adjusts ingredients to the customer; the data scientist adjusts hyperparameters to the data."*

Diagram 6 · p. 24 The per-site manifest in S3 stores the parameters and the last decision. **None** (first training) → conservative defaults; **last approved** → inherit last month's parameters; **last rejected** → targeted search over 10 candidates. All three go through the quality gate, then the model is written and the manifest updated.

Three principles:

- *Stability:* search only when a model failed the gate, so healthy sites don't churn.
- *Continuity:* approved parameters persist month to month.
- *Drift anchor:* defaults are always a candidate, so a site can reset if inherited settings degraded.

The search: 10 candidates, local around the current parameters. Slot 1 is the default (fixed), slot 2 the previous settings (fixed), slots 3–7 one variation each of the five parameters below, and slots 8–10 a second variation of the top three.

Parameter (priority)	Variations	Bounds
learning_rate (1)	×0.5, ×2.0, ×0.8, ×1.25	[0.01, 0.2]
max_depth (2)	−1, +1	[2, 6]
min_child_weight (3)	−3, +3	—
reg_lambda (4)	×0.5, ×2.0	—
n_estimators (5)	−100, +100	—

Flow: generate 10 → train each → evaluate → pick the best → normal quality gate. VERIFY How the variations fill the eight slots, and whether each is perturbed from the previous or the default parameters.

Design point	Reasoning
Bounded cost	10 fits for a rejected site only, not a grid search for every site every month
Priority order	Learning rate and depth matter most for boosted trees; regularization and tree count are second-order
Fixed slots = a floor	The winner is at least as good as default/previous *on the evaluation data*, so tuning can't make a site worse than doing nothing
Bounds	No runaway learning rate or very deep trees

Prepare for: "If you pick the best of 10 on the same data the gate uses, isn't approval optimistic?" VERIFY The good answer: selection used a validation slice and the gate a final holdout. If not, say so and name that fix as a next step.

Resume claim, faster approval for ~60–70% of sites DEFINE For example: "of the sites whose model was rejected, ~60–70% passed after targeted search in the same cycle, instead of waiting for manual re-tuning or another month."

14. Impact: each resume bullet, defended

Claim	Define and measure DEFINE	One-line defense
K8s pipeline for ~450 sites; +80% site-management efficiency	e.g., manual effort per cycle: before, every site triggered and inspected by hand; after, only rejected or flagged sites. Hours per cycle, or share of sites needing a person	"The gate and the approved/rejected split mean people look only at exceptions."
Cold start; +0.23 mean R^2	Sites, truncation months, baseline (§12)	"Gains sit in months 1–2, where new sites were unusable, and healthy sites weren't hurt."
PR drift; ~75% of anomalies caught	Ground-truth list, recall, false-flag rate; backtest of ΔPR flags vs known incidents	"PR divides out size and weather, leaving equipment; flagging stops us training on broken data."
Tuning on rejection; faster approval for ~60–70%	Share of rejected sites approved after search in the same cycle (decision logs, manifests)	"Search runs only where needed, and the fixed slots mean it can't make a site worse."

Claim	Define and measure DEFINE	One-line defense
S3 restructure; −30% data-fetch latency	Stage, mean vs p50, fleet size; before/after on the same run	"Site-first prefixes turn a bucket-wide scan into a direct lookup, across ~450 pods."

If asked for the dollar value: don't invent one. Answer in operating terms: forecast coverage across the fleet, new sites useful from month one instead of month six, analyst and developer time moved from routine runs to exceptions, and fewer bad forecasts reaching decisions. VERIFY A stakeholder's own description of what changed beats all of these.

15. Stakeholders

Keep only the real ones and name one concrete interaction with each; they feed the stories in §23. VERIFY

Stakeholder	What the system needed from them
Manager / team lead	Priorities, thresholds and reviews; acceptable accuracy; policy for sites with no forecast
Platform / DevOps	Cluster access, workflow templates, the Argo CD app, S3 bucket permissions
Data engineering	Snowflake tables; reliable site-spec fields (`is_tracker`, tilt, GCR, kWdc, PIS date) and the active-site list
Asset management / performance engineering	Racking and geometry facts; what counts as an "anomaly" (ground truth for ΔPR)
O&M	Acts on abnormality flags
Forecast consumers / leadership	Whoever reads the central table; the audience for the final presentation

16. Status, limitations and next steps

In production: orchestration, the quality gate, artifact layout, the release process, cold start, adaptive tuning. **Unconfirmed** VERIFY live forecast-error monitoring beyond the training-time gate. **Not built:** probabilistic forecasts and skill metrics. These make strong "what would you improve?" answers.

Area	Limitation	Next step
Evaluation	The gate measures fit, not forecast skill by lead time; R^2 is inflated by the daily cycle	Skill vs a persistence or clear-sky baseline; error by horizon (day 1 vs day 7)
Train/serve inputs	Observed vs forecast weather (§8)	Train on archived forecasts, or report both metrics
Selection bias	Best-of-10 picked on gate data? (§13)	Separate validation slice and gate holdout
PR metric	Average of ratios; curtailment, snow and soiling	Energy-weighted PR, irradiance floor, curtailment flags, threshold tuned on incidents
Pooling	"Similar weather within a state" is weaker in large states	Pool by distance or climate cluster; a hierarchical or global model with site features
Rejected sites	No approved model means no forecast (or a stale one VERIFY)	Explicit, labeled fallback: last approved, pooled model, or clear-sky × PR
Monitoring	The gate runs at training time only	Daily actual-vs-forecast error per site, with alerts
Output	Point forecasts only	Quantile forecasts (P10/P50/P90) for risk-aware consumers
Tuning	Local, coordinate-wise search	Bayesian optimization warm-started from the manifest, same budget

Part 2 — Interview prep

17. Numbers you can say

Say this	Source	Careful phrasing
~450 sites, in production	Resume	"a fleet of about 450 solar sites"
Monthly training, daily inference, hourly forecasts 7 days out	Deck	—
One pod per site, in parallel	Deck	Mention a concurrency cap if one exists VERIFY
Gate: $R^2 \geq 0.80$ and NRMSE ≤ 15	Deck	If unsure about confidentiality: "an R^2 and normalized-error gate"
Resume outcomes: 80% efficiency, +0.23 mean R^2, ~75% anomalies, 60–70% faster approval, −30% fetch latency	Resume	Use the definitions and one-line defenses in §14 DEFINE
K = 500 (~4–5 months); 10 candidates (2 fixed + 8); ΔPR −0.25; routing 1–2 / 3–6 / 6+ months	Deck	Rows → months mapping VERIFY
Backtest: 18 sites in 3 states	Deck chart	"a backtest across 18 sites"

18. Scripts

The 30-second version

> I turned a manual, one-site-at-a-time solar forecasting script into a production Kubernetes pipeline for about 450 sites: monthly parallel training, a quality gate, approved-only daily inference into Snowflake, and GitOps deployment. Then I improved accuracy for new sites with credibility-weighted pooling, which added 0.23 to mean R-squared, flagged equipment problems with a Performance Ratio drift metric that caught about three-quarters of anomalies, and added tuning that runs only when a model is rejected.

"Tell me about your internship" (about 2:45)

> This summer I was an ML Engineering Intern on the Quantitative Analytics team at Cypress Creek Energy, which owns and operates a large solar fleet. The team had a forecasting library that produced hourly forecasts, but it ran one site at a time, by hand, with models saved locally. For about 450 sites, forecast coverage depended on a developer's time. And with no accuracy check, a bad model could quietly feed bad numbers into operational decisions.
>
> I framed the work as four gaps: no automation, no guardrails, poor forecasts for new sites, and static settings. I tackled them in that order, because accuracy doesn't matter if the system can't run reliably or nobody trusts its output.
>
> For automation and trust, I built a Kubernetes-native pipeline on Argo Workflows. Every month it collects the active sites from Snowflake and trains one model per site in parallel pods. Each model must pass an R-squared and normalized-RMSE gate, and is stored in S3 under an approved or a rejected prefix. Daily inference loads only approved models and writes seven-day hourly forecasts to one Snowflake table. A site without an approved model fails in isolation while the rest of the fleet runs. I split orchestration from modeling code and deployed through Argo CD, so a release is a version bump and a push. That replaced manual per-site runs and inspections: about an eighty percent efficiency gain in site management.
>
> Then accuracy. New sites forecast badly for months because they have little history. I pooled data from mature sites in the same state, predicted production as a fraction of capacity, added racking and geometry features,

and weighted the new site's rows so its share of training grows as n over n plus K. That is exactly Bühlmann credibility from actuarial science. In a backtest that treats mature sites as new, mean R-squared rose by 0.23, mostly in the first two months. Some early data is simply broken equipment, so I used the Performance Ratio, which divides out plant size and sunlight, to isolate equipment health. A sharp drop flags the site instead of training on it, which caught about three-quarters of site anomalies. Finally, a rejected model triggers a small, targeted hyperparameter search that always keeps the defaults and last month's settings as candidates. That shortened time-to-approval for sixty to seventy percent of those sites.

It's running in production now. My takeaway: at fleet scale, the hard problems are trust and exceptions — gates, fallbacks and flags — more than squeezing out the last bit of accuracy.

"Walk me through the pipeline" (about 1:45)

There are two scheduled Argo workflows on Kubernetes.

Training runs monthly. It queries Snowflake for all active sites and fans out one pod per site. Each pod pulls weather, generation and site specs, and routes by history. Six-plus months gets the standard per-site model. Three to six months gets the pooled, credibility-weighted cold-start model. One to two months gets a Performance Ratio drift check first, and a sharp drop flags the site instead of training it. Hyperparameters come from a per-site manifest in S3: conservative defaults if there's none, last month's settings if it was approved, and a ten-candidate search if it was rejected. The model is scored on R-squared and NRMSE and written, with its metrics JSON, to the site's approved or rejected prefix.

Inference runs daily, with the same site collection and fan-out. Each pod pulls the weather forecast and site specs and checks the approved prefix. If a model is there, it appends hourly forecasts for the next seven days to one central Snowflake table. If not, that pod fails with the error recorded, and every other site continues.

Deployment is GitOps. Modeling code is a versioned library, orchestration lives in a workflow repo, and Argo CD redeploys whenever that repo changes.

19. Business-decision questions

Q How did you decide what to build first?

By what blocked value. Accuracy is worthless if models don't run reliably or users can't trust the output, so automation and guardrails came first: they made coverage and trust properties of the system, not of someone's time. Then cold start, where forecasts were worst and attention highest. Then tuning, the smallest marginal gain. VERIFY If your manager set this order, "we agreed" is a strong answer too.

Q How did you choose the gate thresholds?

It's coverage versus trust: stricter means more sites without a forecast, looser means bad forecasts reach users. VERIFY The thresholds came from [team convention / the historical metric distribution, so healthy sites pass / agreement with consumers]. I paired R-squared with NRMSE because R-squared alone is flattered by the daily solar cycle, while NRMSE measures error size on a scale comparable across capacities.

Q A rejected site gets no forecast. Isn't that worse than an imperfect one?

I chose explicit failure over silent bad data. Only approved models serve, and a site without one fails visibly while the fleet continues. VERIFY [If true: it keeps its last approved model.] A rejection also triggers the targeted search, which gets most rejected sites approved quickly. What I'd add is a labeled fallback, such as a pooled model or clear-sky times the site's PR, so consumers get something and know it's lower-confidence.

Q Why monthly training and daily inference?

Weather forecasts update daily and consumers want a fresh seven-day view, so inference is daily. History accumulates slowly relative to seasonal change, so monthly training captures new seasons, degradation and new sites at a fraction of the compute. It also gives a natural cadence for the approve-or-reject and tuning decisions.

Q How do you know the work had impact?

Operationally, about 450 sites run without manual work and people only look at exceptions: the roughly 80% efficiency gain [DEFINE]. On the model side, the cold-start, drift and tuning results in §14, each tied to where it mattered. VERIFY A consumer's or manager's quote is the strongest evidence.

Q How did you get buy-in for pooling other sites' data?

With a backtest designed to answer the obvious objection: "won't other sites' data bias this one?" It pretends mature sites are new and draws the pool only from data before the simulated date, so there's no leakage. It showed early gains and no harm to healthy sites. And credibility weighting is easy to explain: the site's own data takes over automatically as it accumulates. VERIFY Say who you presented it to and what they asked.

Q What did it cost to run?

VERIFY Give pod sizing, runtime and concurrency limits. The design keeps cost proportional: search only for rejected sites, capped at ten candidates; monthly rather than daily training; parallel pods that finish quickly instead of one long serial job; and the S3 layout change that cut fetch time.

20. Technical deep dives

Q Explain the cold-start approach with bias and variance.

A new site's own data is small and has no seasonal coverage, so a model trained on it has high variance. Pooling in-state mature sites cuts variance, but adds bias toward other sites. I bought the bias back three ways: normalizing the target by capacity to remove scale differences, adding racking and geometry features to remove structural differences, and upweighting the target's rows so it isn't diluted. Separately, the PR gate removes broken-equipment periods that no model could learn, which effectively cuts irreducible noise.

Q Walk me through the credibility weighting.

Each target row gets weight N over K, where N is the pool size and K is 500. The target's share of total training weight is n times N over K, divided by that plus N, which simplifies to n over n plus K: Bühlmann's credibility factor. So its influence doesn't depend on how many mature sites are pooled, and it grows smoothly with its own data: half the weight at K rows, over 85% by three thousand. K came from domain intuition, about four to five months, and a sweep confirmed it.

Q What is the Performance Ratio, and why use it for drift?

Production divided by capacity times plane-of-array irradiance in suns. Dividing by capacity removes plant size, and dividing by irradiance removes weather, so what's left is how efficiently the equipment converts available sunlight. For young sites, I split history 80/20 chronologically and compare test PR with train PR. A drop worse than minus 0.25 means something broke recently, so the site is flagged instead of trained on. It's one-sided on purpose: getting healthier is fine to train on; getting worse can't be modeled.

Q Why R^2 and NRMSE rather than MAPE?

> MAPE blows up near zero, and solar is zero every night and near zero at dawn and dusk. R-squared captures explained variance but is flattered by the daily cycle. NRMSE gives error magnitude in a normalized unit that's comparable across sites. VERIFY State your normalizer. Next I'd add skill against a persistence or clear-sky baseline, and error by forecast horizon.

Q Why XGBoost?

> It's the strongest general-purpose choice for tabular, nonlinear relationships like irradiance, temperature and time of day against output. It trains fast enough for one model per site across 450 sites monthly, supports sample weights natively, which credibility weighting needs, and has well-understood regularization knobs, which are exactly what the tuning searches over. The defaults are deliberately conservative.

Q How did you make the backtest leakage-safe?

> Two ways. The target is truncated to its first N months, and the pool comes only from a two-year window ending at the target's truncated last date, so nothing from after the simulated moment enters training. And the PR check splits chronologically, not randomly, because adjacent hours are highly correlated and a random split would leak.

Q Why Argo Workflows rather than Airflow?

> VERIFY Adjust to what was actually decided. Argo is Kubernetes-native: every step is a container in its own pod, so per-site fan-out, resource isolation and failure isolation come naturally, and artifacts go straight to S3. It fits a team already running Kubernetes and Argo CD. Airflow is excellent for broader data scheduling, but it would add a separate scheduler and workers to operate.

Q How does the targeted search work, and why is it safe?

> Ten candidates. Two are fixed: the defaults and last month's settings. Eight are one-at-a-time perturbations in priority order: learning rate, max depth, min child weight, lambda, number of trees. Each stays within bounds; learning rate stays within 0.01 to 0.2 and depth within 2 to 6. I train all ten, pick the best, and run it through the normal gate. The fixed slots guarantee the result is no worse than doing nothing on the evaluation data, and the defaults act as a reset anchor if inherited settings drifted.

21. Quick answers: "How did you...?"

Question	One-breath answer
...keep a bad model from reaching users?	Structurally: training writes each model to approved or rejected by threshold and inference reads only approved, so a rejected model can't load and a site without one fails visibly.
...stop one site breaking the fleet?	One pod per site; a failure kills only that pod, with the error recorded, while the fan-out completes. VERIFY Retries or `continueOn`?
...make runs reproducible and traceable?	joblib model + metrics JSON per run; library pinned by version in the workflow repo; GitOps so the cluster matches Git; the manifest records each site's settings and decision.
...onboard a new site?	Nothing manual. Site collection picks it up; routing goes PR check → cold start → standard as history grows; tuning starts from defaults.
...explain it to non-experts?	Four business gaps, shown closing; the chef analogy for tuning; PR as "the share of sunlight the site turns into electricity." VERIFY Name the audience.

22. Tough follow-ups

Q You were an intern. What did *you* build, versus what already existed?

> VERIFY Be exact. Existing: the forecasting library (core preprocessing, features, model) and the Snowflake data. Mine: [the Argo training and inference workflows, the guardrails and S3 layout, the repo split and Argo CD setup, the cold-start pipeline, PR drift, and adaptive tuning]. Name anything a teammate built or co-built. Precision here builds trust.

Q How exactly is the 80% efficiency calculated?

> Have the formula ready (§14) DEFINE. If it's an estimate, for example the share of sites that no longer need manual handling, say it's an estimate and how you derived it.

Q R^2 of 0.8 on hourly solar isn't impressive. The sun explains most of it.

> Agreed, which is why it isn't the only check. NRMSE bounds the error magnitude. The next step I'd add is skill against a clear-sky or persistence baseline, and error by forecast horizon.

Q Did you pick the best tuning candidate on the same data the gate uses?

> VERIFY Answer truthfully. If yes: "Yes, which makes approval somewhat optimistic. The fix is a separate validation slice for selection and a final holdout for the gate."

Q Isn't "similar weather within a state" a big assumption?

> Yes. It's the main assumption, and it's weakest in large or climatically diverse states. The fixes are pooling by distance or climate cluster, and conditioning on local weather features, which the model already does. The backtest covered IL, NC and OR.

Q Your training uses observed weather, but inference uses forecasts?

> VERIFY If true: "Then the gate measures modeling error, not operational error, which also includes forecast error. Training on archived historical forecasts, or reporting both metrics, fixes that." If you already used historical forecasts, say so; it's a strength.

Q What happens with curtailment or snow?

> Both lower production without equipment failure, so they can produce false PR flags and noisy training data. A curtailment flag from operations data, plus seasonal handling, is the fix. VERIFY Say what exists today.

Q Why a model per site instead of one global model?

> Per-site models capture site-specific behavior once there's enough history, and failures are isolated per site. Cold start is effectively a partial global model. The natural evolution is one global model with site features and credibility-style weighting for every site: less operational surface, more modeling complexity.

Q What would you do differently?

> Build live forecast-error monitoring earlier, because the gate runs at training time and actual-vs-forecast monitoring shows real-world drift. And agree an explicit fallback policy for rejected sites with the forecast consumers up front.

23. Behavioral story bank STORY

The technical content comes from the deck; everything in [brackets] must be your real detail. If a scaffold didn't happen, delete it rather than adapt a story you can't back up. Aim for 60–90 seconds and end with the result and a lesson.

Which story answers which question: cross-functional → S1, S2 · disagreement → S4 · ambiguity → S3 · failure → S5 · learning fast → S6 · non-technical audience → S7 · prioritization → S8 · ownership → S9 · influence without authority → S10 · feedback → S11 · proudest / biggest impact → S1 + S10.

#	Story	Situation and action (your details in brackets)	Result → lesson
S1	Production with platform / DevOps	Needed cluster access, templates, S3 permissions and an Argo CD app [owned by platform]; you proposed the library/workflow split to keep their surface small. [How you engaged; conventions adopted; tested on a few sites first.]	No hand-deploys [what it saved them] → *design for the team that operates it.*
S2	Site data with asset management	Needed racking, tilt/GCR, kWdc and PIS dates, plus an anomaly ground truth. [Who you asked; how you translated; data issues found, e.g., a wrong capacity; validating flags against their incident list.]	Features in production; ~75% measured against [their list] → *domain experts are the fastest route to features and ground truth.*
S3	Choosing K and the ΔPR threshold	No labeled answer: anchored K on ~4–5 months and swept it [values, metric]; one-sided −0.25 with a clear rationale [who checked it].	Documented, defensible parameters → *domain prior, backtest, write down why.*
S4	Fail loudly or always serve?	[Someone] wanted a forecast for every site; you argued for approved-only. [Your framing, the data, the compromise, e.g., last approved model or a labeled fallback.]	[Outcome] → *disagree with data and options; find a design that serves both.*
S5	Dilution in pooled training	[If true:] unweighted pooling didn't help as mature sites were added; you diagnosed dilution and derived n/(n+K).	Weighting shipped [metric change] → *when more data hurts, check whose data dominates the loss.*
S6	Learning Argo / K8s / Argo CD fast	[Plan: docs → minimal workflow → one site end to end → fan-out; who helped.]	[Time to first production run] → *one thin slice end to end before scaling.*
S7	Final presentation	Four gaps; the chef analogy; PR as sunlight turned into electricity; per-site before/after lines.	[Questions asked, follow-ups, adoption.]
S8	Automation before accuracy	Ordered by what blocked value [how you agreed it; what you deferred].	Production-ready early, improvements layered on.
S9	S3 layout or release process	[Noticed while building something else; proposed; migrated existing artifacts.]	−30% fetch latency DEFINE or one-push releases.
S10	Getting pooling adopted	Anticipated the bias objection; leakage-safe backtest; showed no-change sites too.	[Adopted into production.]
S11	Feedback	[Real feedback on code review, the deck or scope; what you changed.]	[Result.]

24. The forward-deployed lens and delivery

> Fleet-scale ML is an exceptions-management problem. Automate the routine path, gate on quality, isolate failures, give new assets a principled cold start, and use a domain metric like PR to tell data problems from model problems.

Discovery questions for an energy or industrial customer:

1. **Decisions:** who consumes the forecast, for which decision (O&M dispatch, revenue, market bidding, storage dispatch), and at what horizon and granularity?

2. **Error economics:** is over- or under-forecasting worse? Is a point forecast enough, or do they need P10/P50/P90?
3. **Data:** where are metering, weather and metadata (capacity, racking, commissioning dates)? How reliable is the metadata? Where is curtailment logged?
4. **Ownership:** who acts on a flag, and who owns a site without a forecast? Is a fallback acceptable?
5. **Onboarding:** how many new assets arrive each year? That says how much cold start matters.
6. **Platform:** Kubernetes and cloud standards, data residency, and the release process their platform team will accept.

Why this maps to forward-deployed work: balancing a platform team's constraints with a modeling goal; turning domain experts' knowledge into features and ground truth; making a system trustworthy for people who won't read the code.

Delivery tips (one-way video):

- **Business first.** Open every answer with the operational problem ("coverage depended on a developer's time") before the technology.
- **Name the trade-offs:** coverage vs trust, specificity vs data volume.
- **Numbers with definitions:** "plus 0.23 mean R^2, in a backtest simulating new sites, mostly in the first two months."
- **"I" vs "we":** "I built" for your work, "the team had" for what existed. Interviewers listen for it.
- **Behavioral answers:** 60–90 seconds, STAR, end with the lesson. Stay generalized (Appendix B).
- **Timing:** the 2-minute script runs about 2:45 and the walkthrough about 1:45. Trim the tuning sentence first.

Appendix

A. Pre-interview verification checklist

Write your answers in the right-hand column. Anything you can't confirm, drop from your scripts.

#	Numbers to define DEFINE	Your answer
1	80% efficiency: the formula, and before/after values	
2	+0.23 R^2: which sites, which months, mean of what; the baseline	
3	~75% anomalies: ground-truth source, count, false-flag rate	
4	~60–70%: the denominator (rejected sites?) and what "time-to-approval" means	
5	−30% latency: which stage, mean or p50, fleet size	
6	Share of sites passing the gate in a typical month	
7	Runtime of a monthly training run and a daily inference run	
8	K sweep: values tested, metric, result	

#	Facts to confirm VERIFY	Your answer
1	Who consumes the Snowflake forecast table, and for what (§2)	
2	Argo mechanics: CronWorkflow, `withParam`, `when`, `parallelism`, `retryStrategy`, `continueOn` (§3, §21)	
3	Contents of the metrics JSON (§4)	
4	Rejected site: last approved model or no forecast? Idempotent append? (§4)	
5	NRMSE normalizer; how thresholds were chosen; evaluation split (§5)	
6	The S3 layout before the restructure (§6)	
7	Where the package is published; CI before publishing? (§7)	
8	XGBoost? Feature list, degradation, standard-pipeline target; observed vs forecast weather in training (§8)	
9	`tilt_gcr` one feature or two; rows → months for K = 500 (§10)	
10	PR: average of ratios or ratio of sums; irradiance floor; curtailment; why only 1–2 months; later extended? (§11)	
11	Backtest baseline; what the IL dips were (§12)	
12	How variations fill slots; perturbed from what; selection data vs gate data (§13)	
13	Does live forecast-error monitoring exist? (§16)	
14	Exactly what you built vs what existed or teammates built; why Argo (§20, §22)	

Stories: fill in S1–S11 (§23). At minimum have cross-functional, disagreement, failure, ambiguity, and explaining-to-non-experts ready.

B. Confidentiality: what to generalize

The deck is marked *Proprietary and Confidential*. Your resume numbers are already public (you wrote them), so they're fine to say.

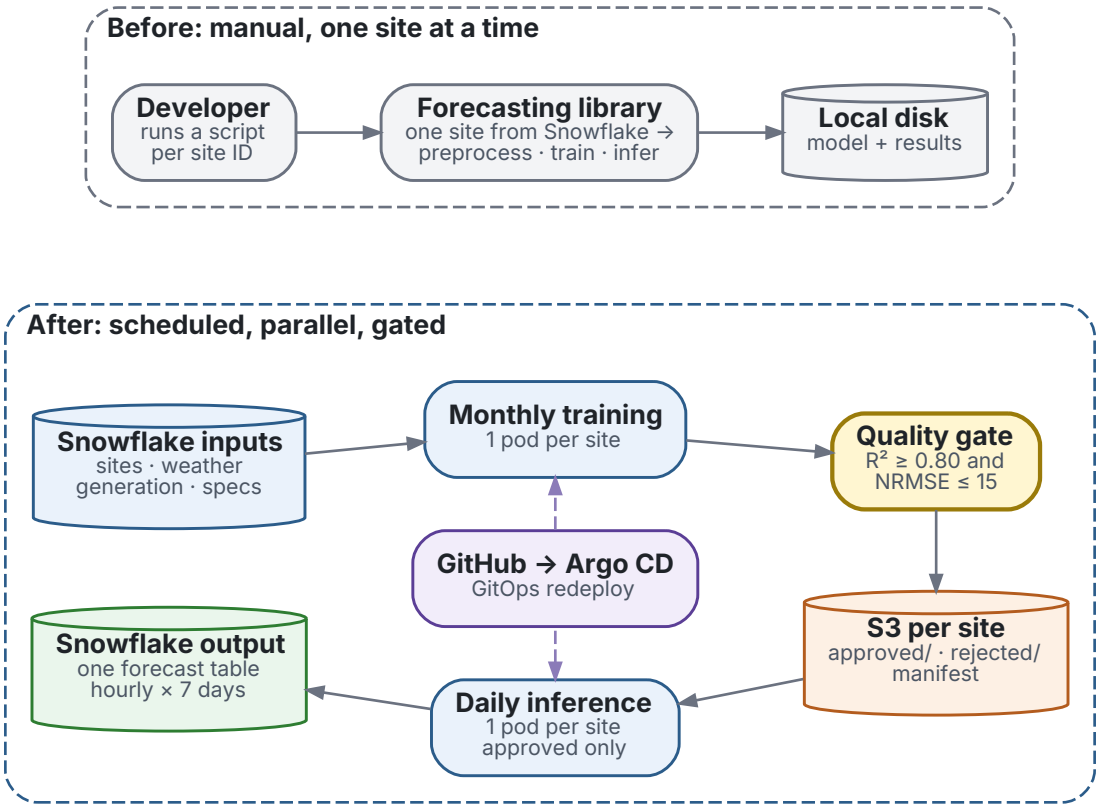
Item	Instead of…	Say…
Site IDs	"OR-000180," "IL-003326"	"a site in Oregon," "one Illinois site"
Internal repo names	"ra-lib-forecast," "ra-wf-solar-forecast"	"the forecasting library," "the workflow repo"
Tables and buckets	Actual names	"a central Snowflake forecast table," "an S3 bucket per environment"
Exact thresholds / params	Usually fine at this level; if unsure…	"an R^2 and normalized-RMSE gate," "a credibility constant of a few hundred rows"
Fleet composition	Counts by state, or capacities	"about 450 sites across several states"
Charts and slides	Screen-sharing the deck	Describe results verbally; never show confidential slides in a recording
Business outcomes	Revenue figures you may have heard	Operational impact only (§14)

When in doubt, one sentence covers it: *"I'll keep this at a level that's appropriate for company-confidential work."* Interviewers respect it.

Diagrams

Every diagram referenced in the text, in order. Each one names the section that describes it and links back to that page.

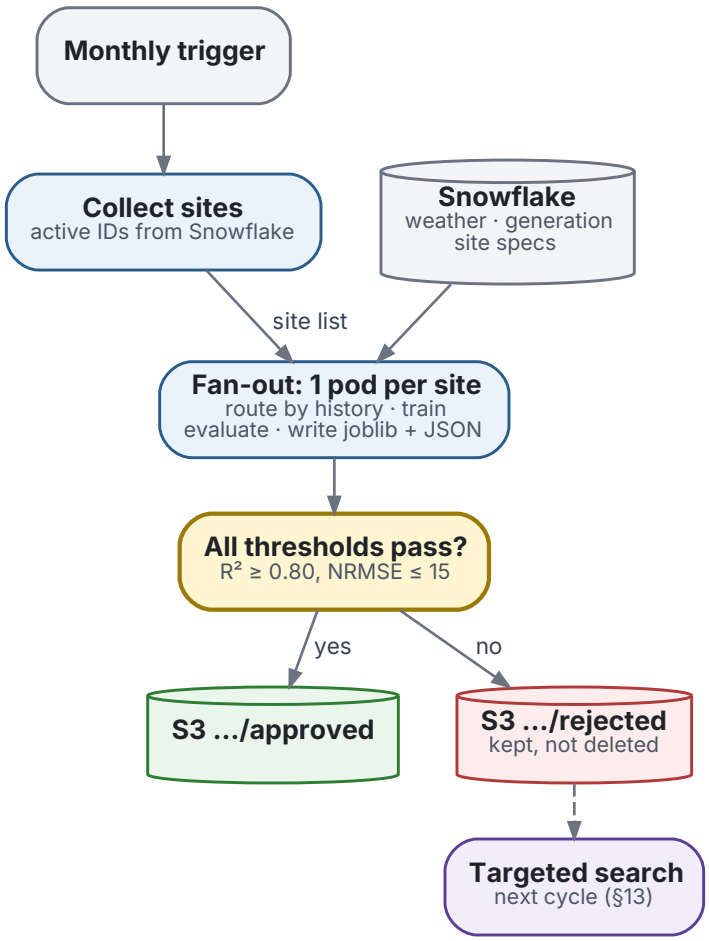
Diagram 1 · System architecture: before and after



Dashed purple lines are deployment, not data flow.

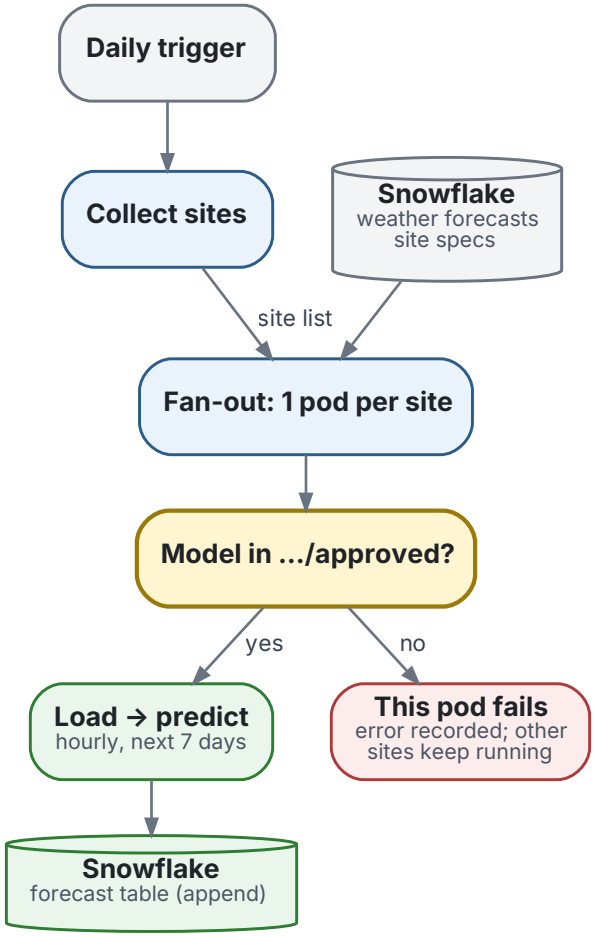
Described in §3 Architecture, page 2

Diagram 2 · Training pipeline (monthly)



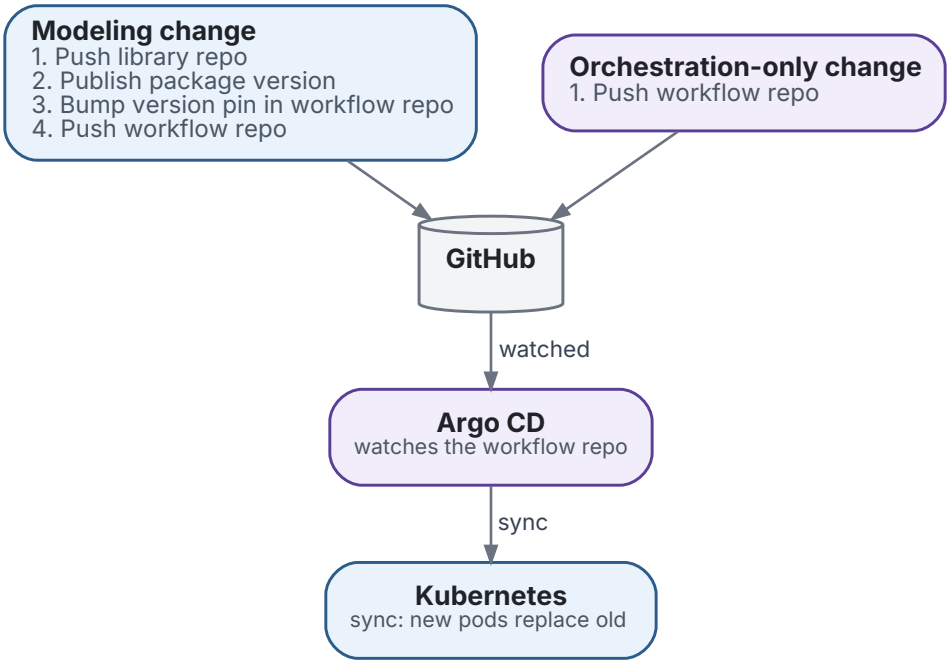
Described in §4 The two pipelines, page 3

Diagram 3 · Inference pipeline (daily)



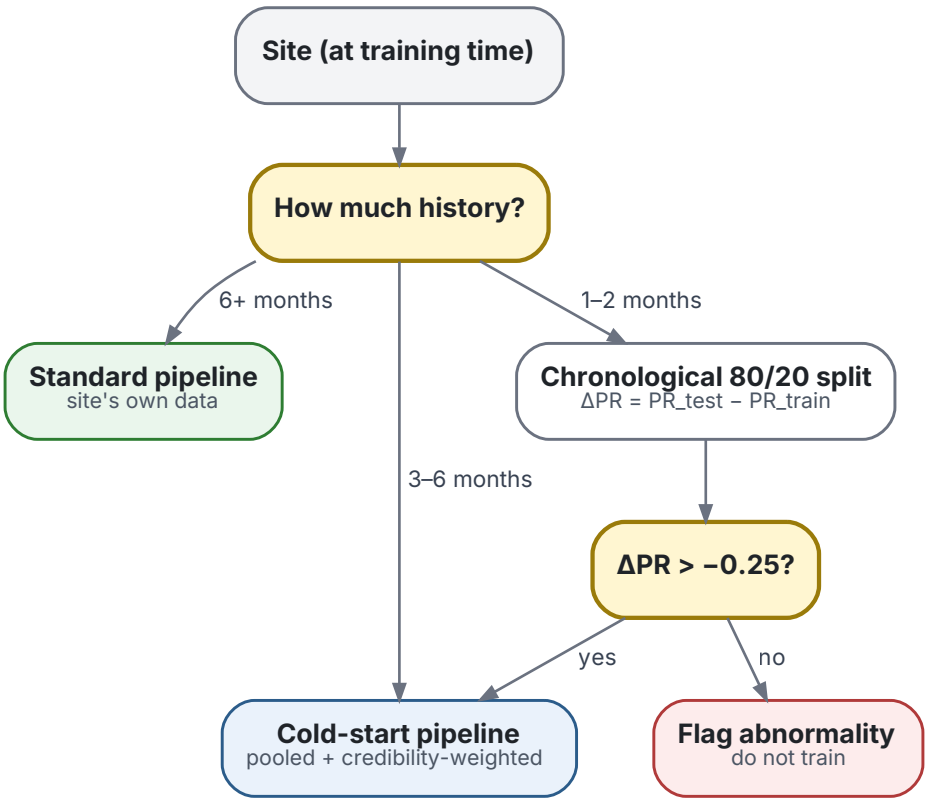
Described in §4 The two pipelines, page 3

Diagram 4 · Release flow: repo split + Argo CD



Described in §7 Release engineering: repo split + Argo CD, page 4

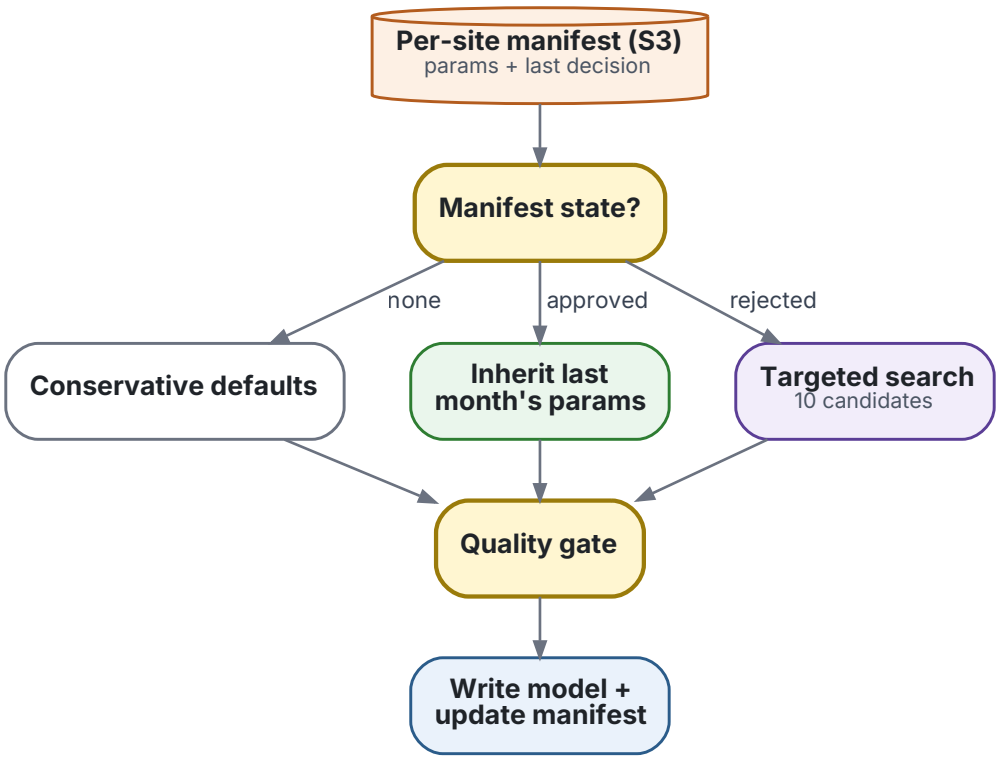
Diagram 5 · Routing by data availability



Every site is routed at training time.

Described in §11 Performance Ratio drift: isolating equipment health, page 6

Diagram 6 · Adaptive tuning: the manifest decision



Described in §13 Adaptive hyperparameter tuning, page 7